Laboratory report of Nanjing University of Information Science and Technology

Experiment Title _Cloud Image Registry & Distribution_ Data _2026/6/6_
Class __Applied Computing__ Name __Wang Shiyu__ ID __202383910020__

一、Experimental purpose

Practice Version Control: Learn to use Docker Tags to manage different iterations of a cloud application.
Validate Decoupled Deployment: Simulate a disaster recovery scenario to verify the ability to deploy services in a remote environment without source code.
Security Best Practices: Implement Secret Management using Personal Access Tokens (PAT) instead of plaintext passwords.

二、Experiment Contents

In this lab, you will transition from a "Standalone Developer" to a "Cloud Engineer." You will:

- Configure secure authentication with GitHub Container Registry (GHCR).
- Tag and push your web application image to the cloud.
- Perform a version upgrade (v1.0 to v2.0) and observe the Layered Storage mechanism.
- Perform a "Clean - room Deployment" by pulling your image onto a fresh environment after wiping local data.

三、Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. Screenshots must demonstrate that they were taken on your own computer, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Secure Authentication (PAT Generation)

Cloud platforms use Tokens for automation and security.

1.1 Generate Token

Navigate to GitHub Settings -> Developer settings -> Personal access tokens -> Tokens (classic).

1.2 Permissions

Click Generate new token (classic). Select the scope write:packages (this automatically includes repo and read:packages).

1.3 Save Token

Copy the generated token immediately. You will not be able to see it again.

1.4 Login via Terminal

In your Codespaces terminal, run and explain the following code:

export CR_PAT=YOUR_COPIED_TOKEN

echo $CR_PAT | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin

Observation: You should see Login Succeeded.

Note: You may see a warning about "credentials stored unencrypted." This is normal in a Cloud IDE environment like Codespaces. As long as you see "Login Succeeded," you are ready for Step

Step 2: Tagging for the Cloud

Docker images require a specific naming convention to be recognized by cloud registries.

2.1 Build the Docker Image cloud-lab-image:v1 based on lab 2.

2.2 Apply Tag

Convert your local image into a cloud-ready format.

Note: Your GitHub username must be in LOWERCASE

docker tag cloud-lab-image:v1 ghcr.io/YOUR_USERNAME/cloud-app:v1.0

Step 3: Pushing the Image (The "Ship" Phase)

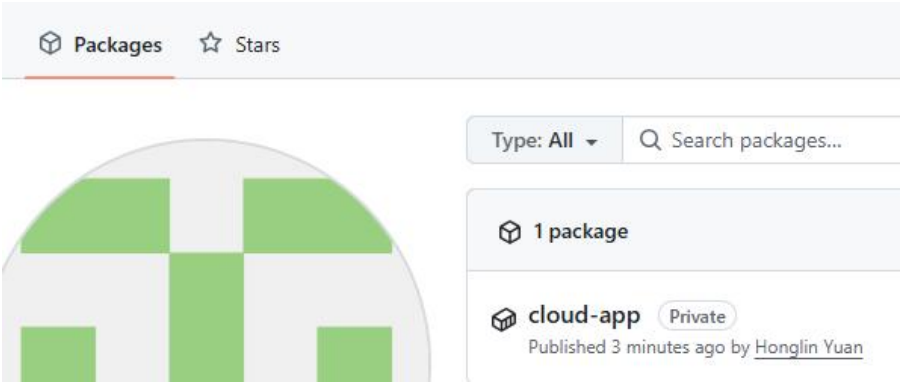
Upload your packaged application to GitHub's global infrastructure.

3.1 Push

docker push ghcr.io/YOUR_USERNAME/cloud-app:v1.0

3.2 Verify

Go to your GitHub Profile -> Packages. You should see cloud - app listed.



Step 4: Iteration & Layer Analysis (v2.0)

Cloud computing relies on incremental updates.

4.1 Modify Code

Open index.html and change a visible element (e.g., change the background color or update text to "Welcome to Version 2.0").

4.2 Rebuild & Retag

docker build -t cloud-lab-image:v2 .

docker tag cloud-lab-image:v2 ghcr.io/YOUR_USERNAME/cloud-app:v2.0

4.3 Push v2.0

Run docker push ghcr.io/YOUR_USERNAME/cloud-app:v2.0

Observation: Notice that some lines say Layer already exists. This is Docker's efficiency—only the changed "layer" (the HTML) is uploaded, saving time and bandwidth.

Step 5: Disaster Recovery (The "Pull" Test)

Prove that your application is now independent of your local machine.

5.1 Wipe Local Data

Delete all local images and containers to simulate a new server.

```
# Warning: This removes all local images
docker system prune -a
```

5.2 Remote Deployment

Deploy directly from the cloud without any source code files.

```
docker run -d -p 8888:80 ghcr.io/YOUR_USERNAME/cloud-app:v1.0
```

5.3 Verify

Access the service via the "Ports" tab on port 8888.

5.4 Run

docker run -d -p 8888:80 ghcr.io/honglinyuan/cloud-app:v2.0

You will see Bind for 0.0.0.0:8888 failed: port is already allocated. Find methods to solve this problem.

Step 6: Image Optimization - "Standard vs. Alpine" (15 mins)

In cloud computing, image size directly impacts storage costs and deployment speed (Cold Start time).

6.1 Compare Base Images

Create two different Dockerfiles to see the impact of OS selection:

6.1 Create a file named Dockerfile.alpine with the following content

FROM nginx:alpine

COPY . /usr/share/nginx/html

6.2 Build both versions

docker build -t cloud-app:standard -f Dockerfile .

docker build -t cloud-app:alpine -f Dockerfile.alpine .

6.3 Size Comparison

Run : docker pull nginx:latest

docker pull nginx:alpine

docker images | grep nginx

compare the size of cloud - app:standard vs. cloud - app:alpine.

6.4 Report Task

Record the exact sizes in your lab report. Explain why "Alpine" is the industry standard for production microservices.

Step 7: Use git to save your changes

- git add . (Gather all your lab changes)
- git commit -m "Finished Lab 03 " (Seal the package)
- git push (Ship it to GitHub for permanent storage)

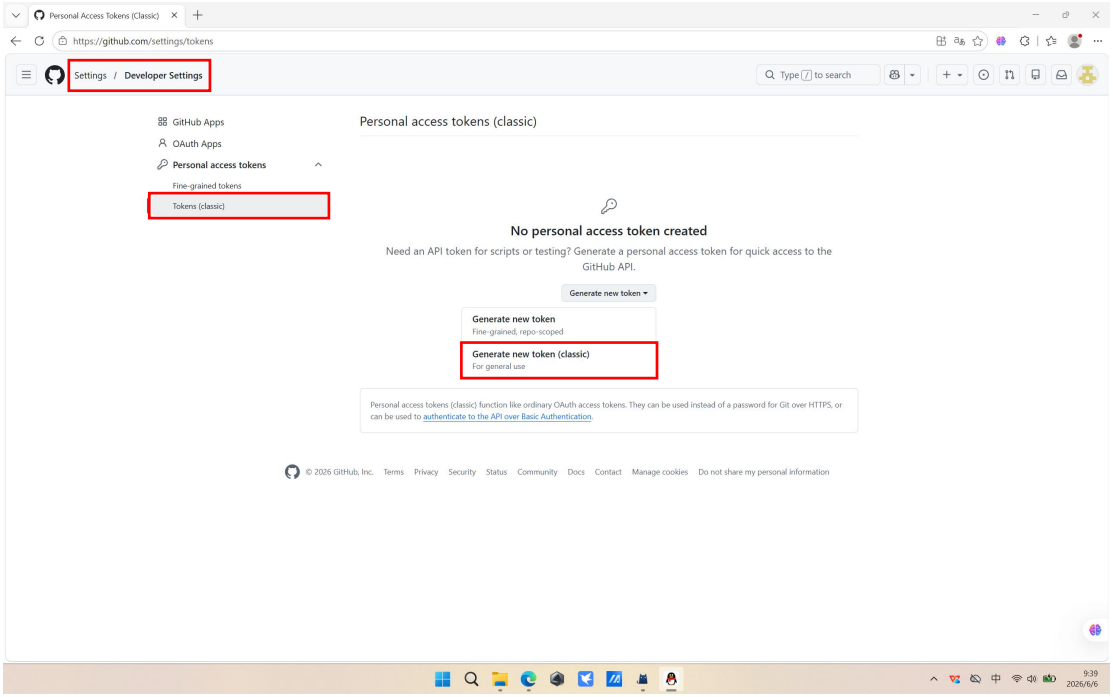
四、Experiment Process

Step 1: Secure Authentication (PAT Generation)

1. Generate Token

I navigated to GitHub Settings to generate a Personal Access Token (PAT) for secure authentication with GitHub Container Registry (GHCR).

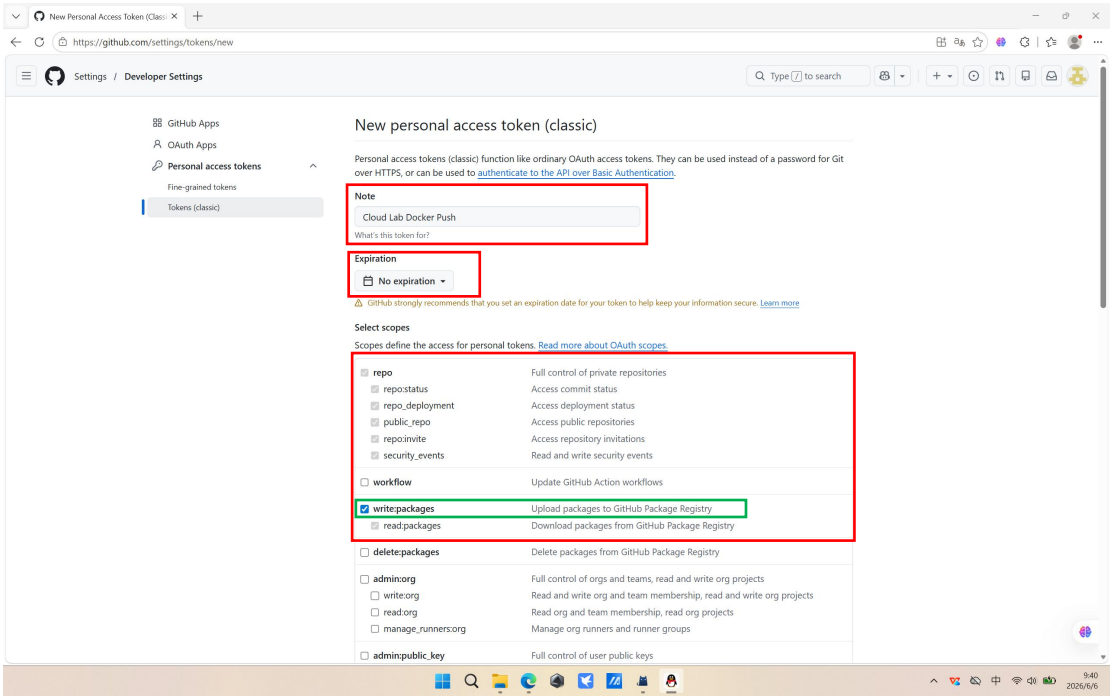
1) Logged into my GitHub account
2) Clicked on profile avatar → Settings
3) Located Developer settings at the bottom of the left sidebar
4) Selected Personal access tokens → Tokens (classic)
5) Clicked Generate new token (classic)



2. Configure Permissions

On the token generation page, I configured the following settings:

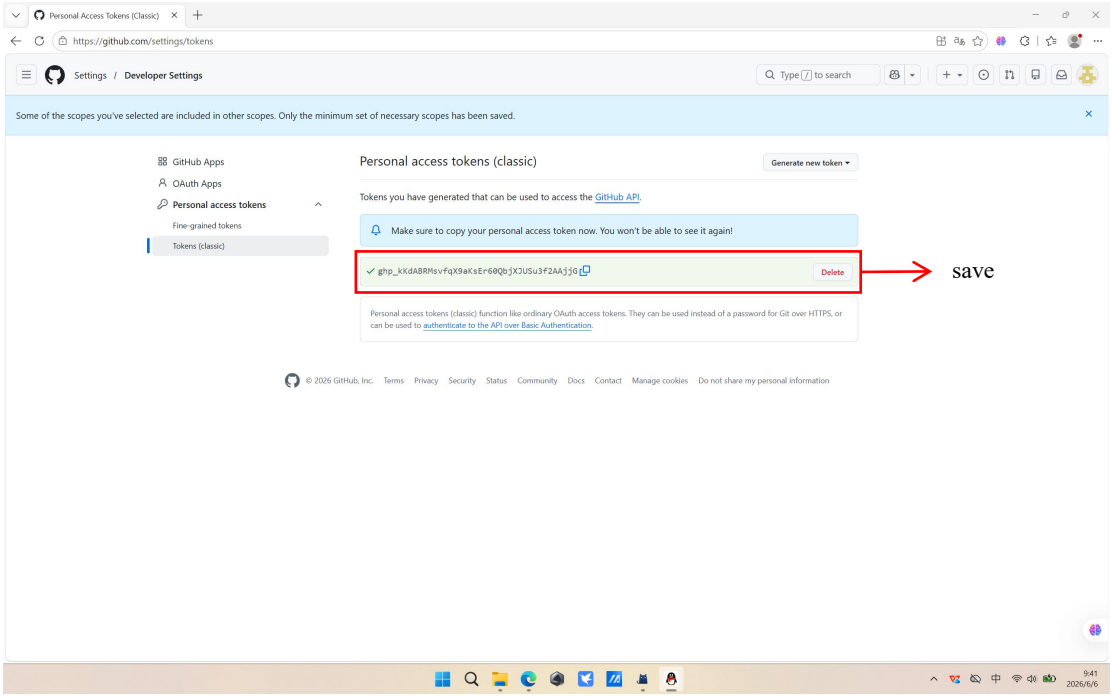
- Note: Cloud Lab Docker Push
- Expiration: No expiration
- Scopes: Selected write:packages, which automatically includes repo and read:packages permissions



3. Save Token

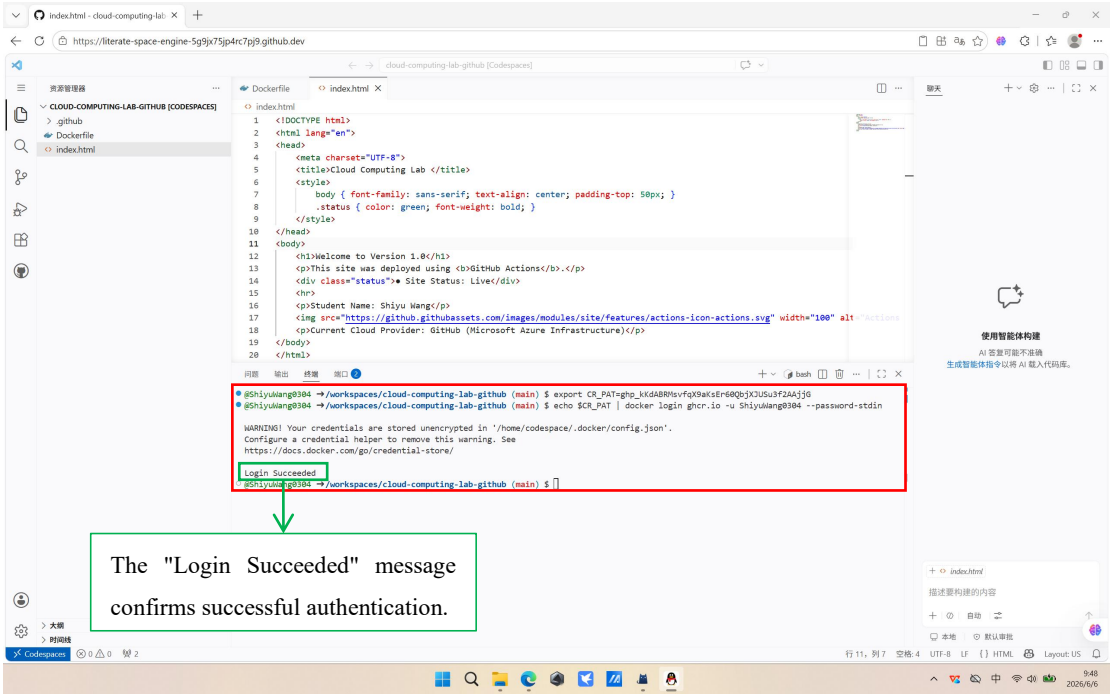
After clicking Generate token, I immediately copied the generated token to a secure location. The

token starts with ghp_ prefix.



4. Login via Terminal

In the Codespaces terminal, I executed the following commands to authenticate with GHCR.

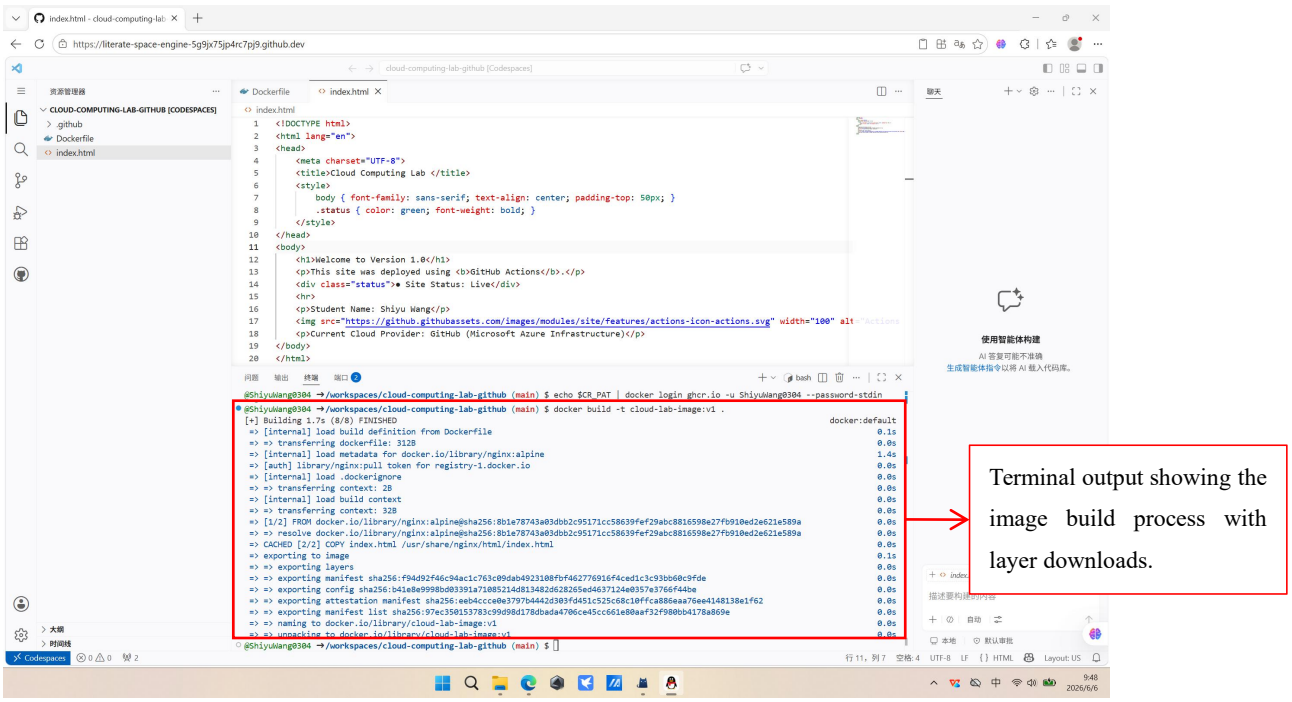


- export CR_PAT: Stores the token as an environment variable, preventing plaintext password exposure in command history
- echo $CR_PAT | : --password-stdin: Passes the password via standard input, which is more secure than using -p PASSWORD
- ghcr.io: GitHub Container Registry endpoint

Step 2: Tagging for the Cloud

1. Build the Docker Image

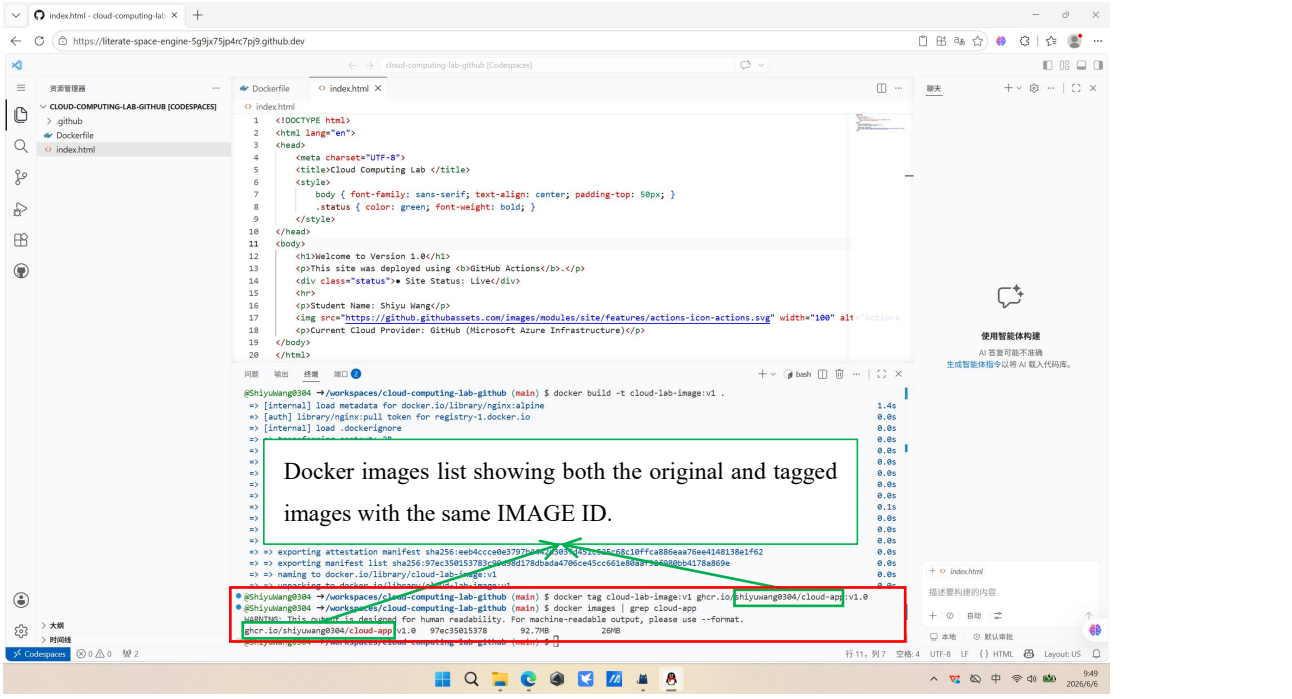
Based on Lab 2, I built the v1 version of the Docker image.



2. Apply Tag

I converted the local image to a cloud-ready format by applying the GHCR naming convention.

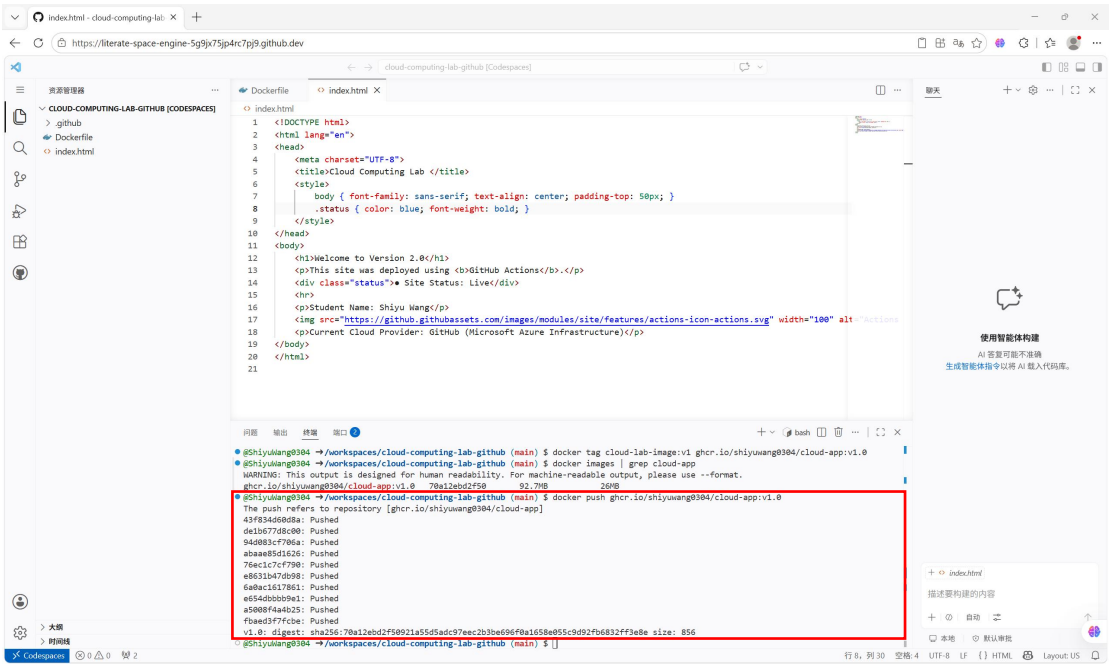
- ghcr.io/ - GitHub Container Registry hostname
- shiyuwang0304/ - My GitHub username (must be lowercase)
- cloud-app - Package/image name
- v1.0 - Version tag



Step 3: Pushing the Image (The "Ship" Phase)

1. Push to GHCR

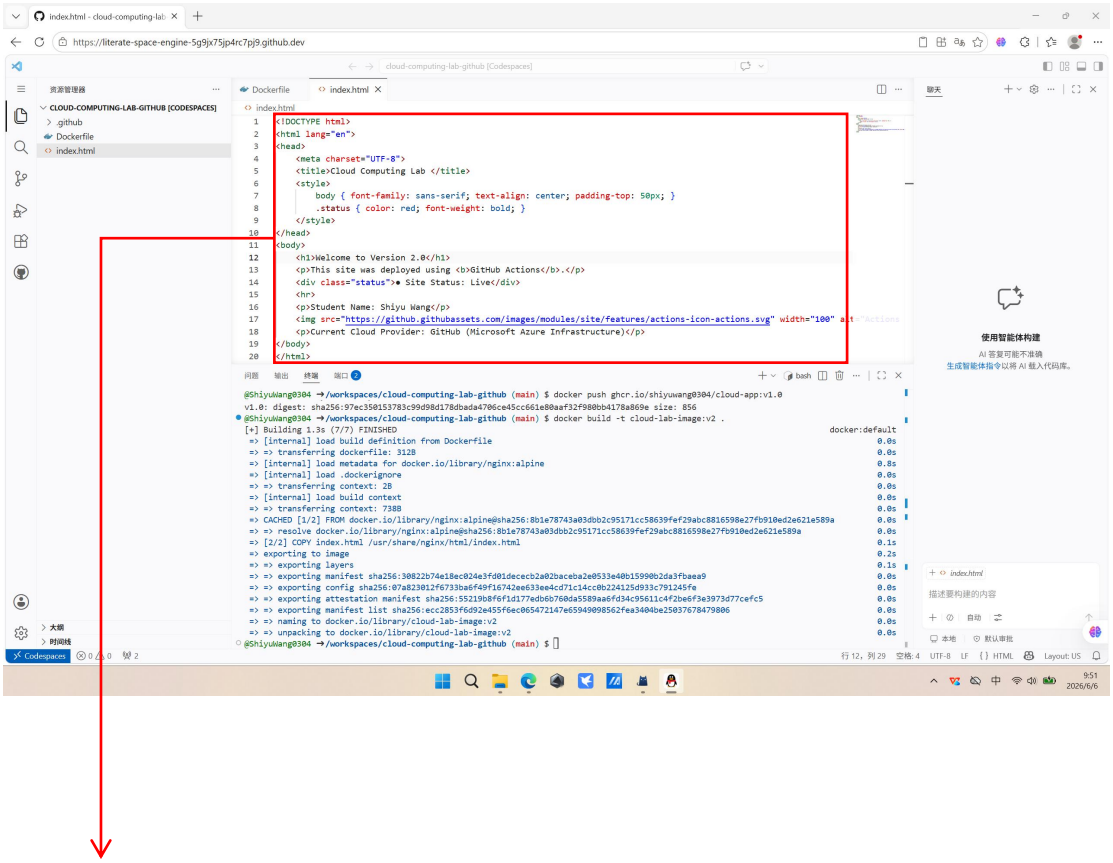
I uploaded the containerized application to GitHub's global infrastructure.



Step 4: Iteration & Layer Analysis (v2.0)

1. Modify Code

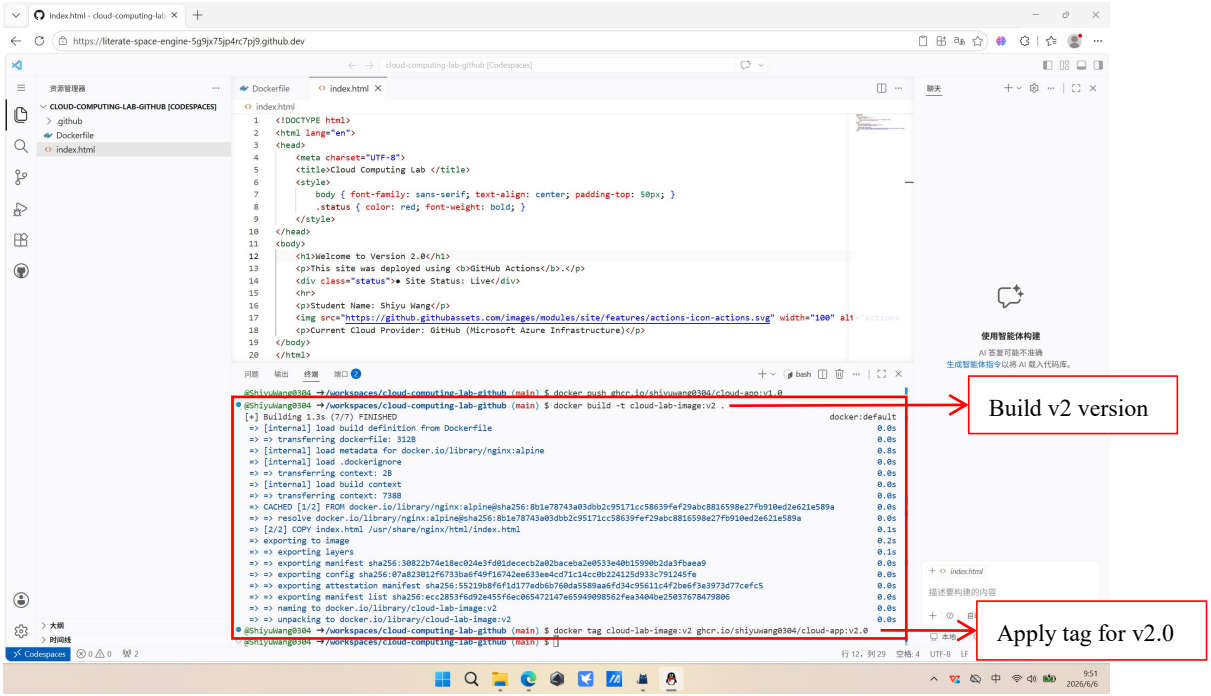
I opened index.html and made a visible change to demonstrate version control.



Version 1.0 Version 2.0

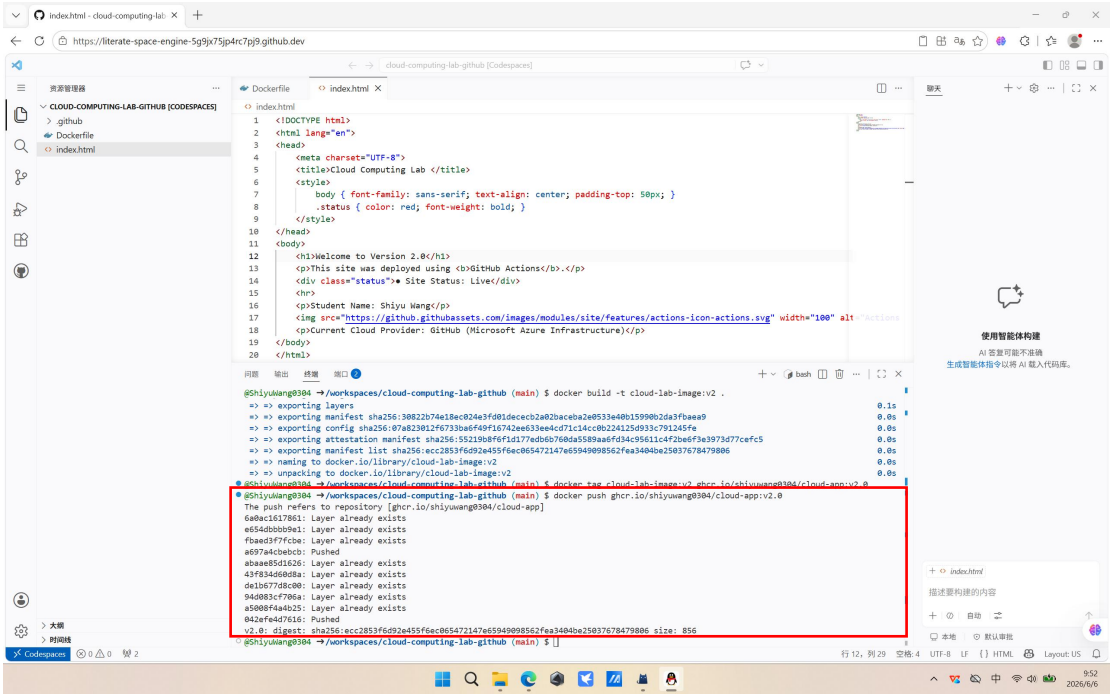
2. Rebuild & Retag

I rebuilt the image with the new changes and applied a new tag.



3. Push v2.0

I pushed the new version to GHCR.



Docker Layered Storage:

During the push of v2.0, I observed that some layers showed "Layer already exists" while others were pushed. This demonstrates Docker's efficiency through layered storage.

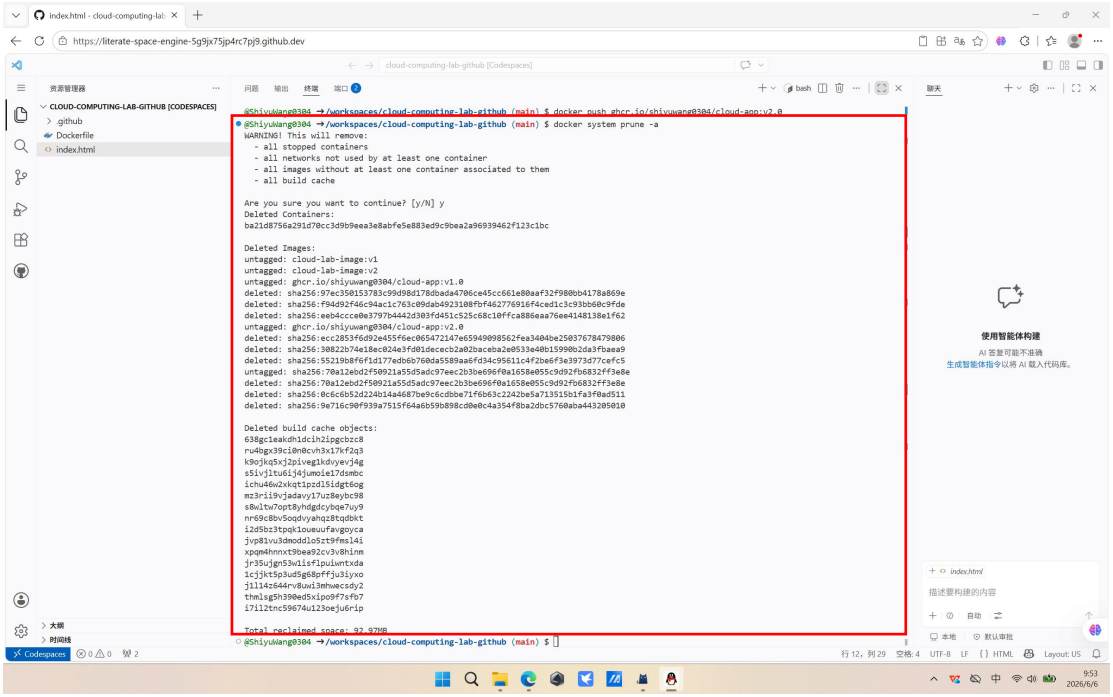
Layer Type	v1.0 Push	v2.0 Push	Reason
Nginx base image	Uploaded	Layer already exists	No changes to base OS
System libraries	Uploaded	Layer already exists	No changes to dependencies
Application files	Uploaded	Pushed	index.html was modified

Step 5: Disaster Recovery (The "Pull" Test)

This step proves the application is independent of the local development environment and can be deployed from the cloud registry alone.

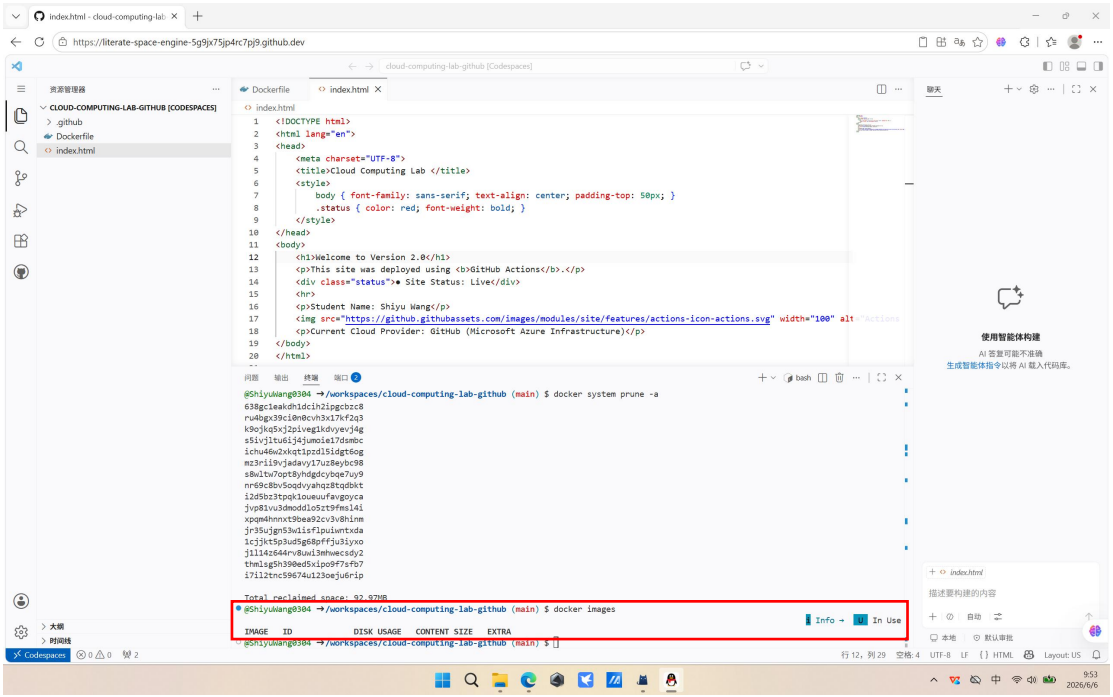
1. Wipe Local Data

I simulated a fresh server environment by removing all local Docker artifacts. Terminal showing the prune command execution and confirmation prompt.



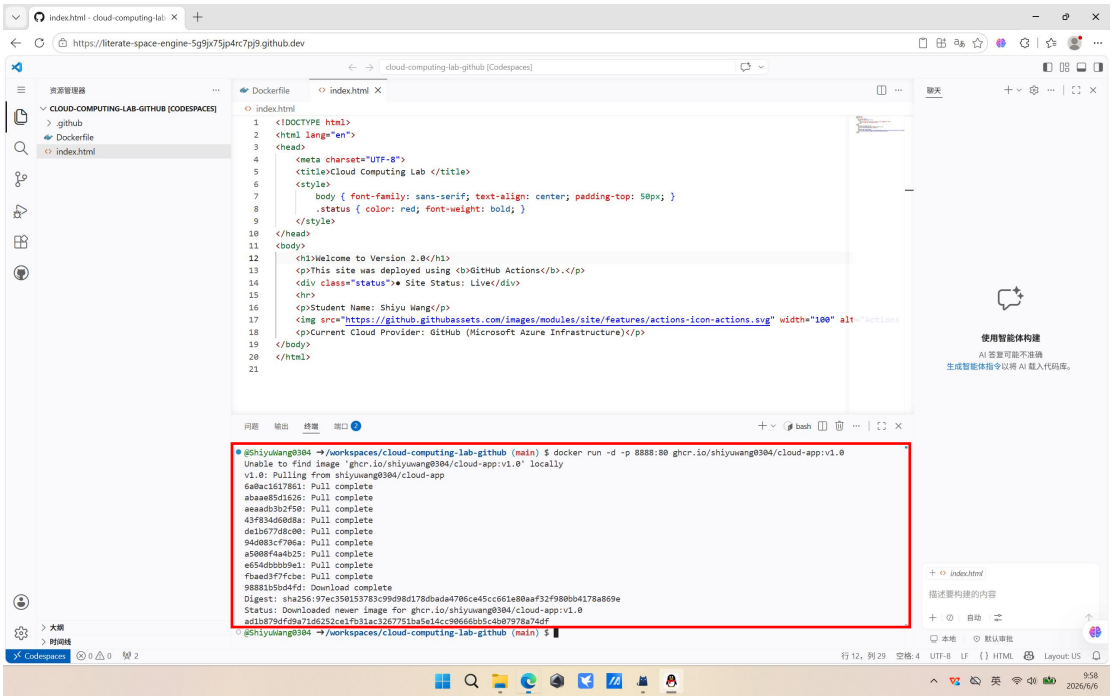
2. Verify Local Cleanup

I confirmed all images were removed. Empty image list confirming successful cleanup.



3. Remote Deployment (Clean-Room Deployment)

I deployed the application directly from GHCR without any source code files. Container successfully started with the pulled image.

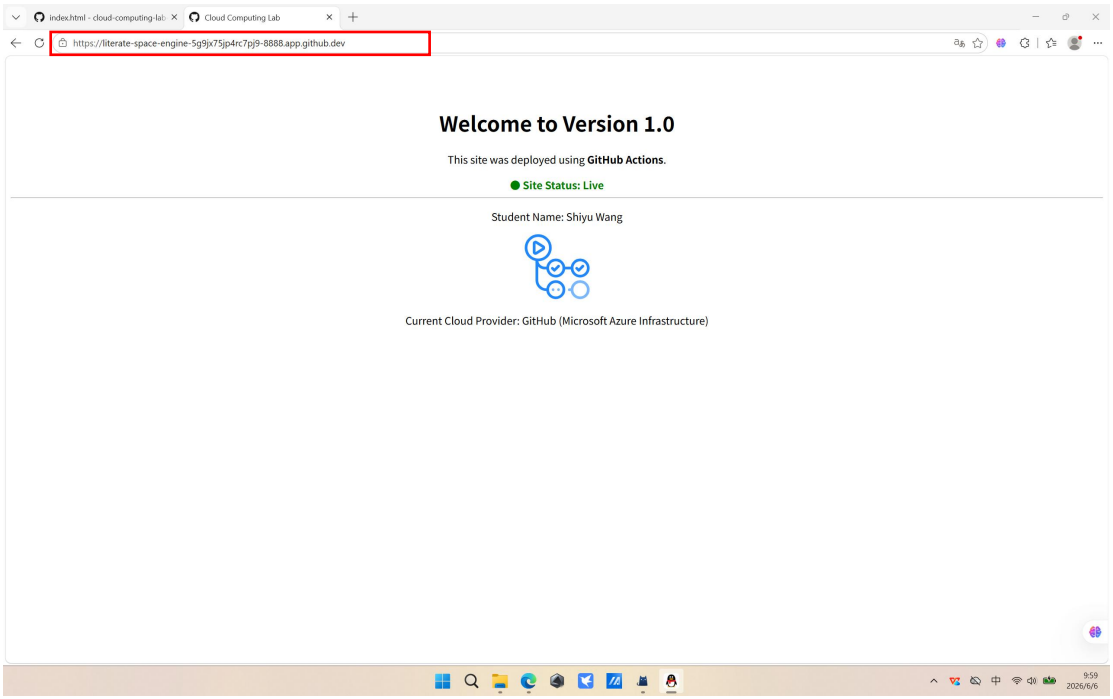


4. Verify Service

I accessed the application via the Codespaces Ports tab:

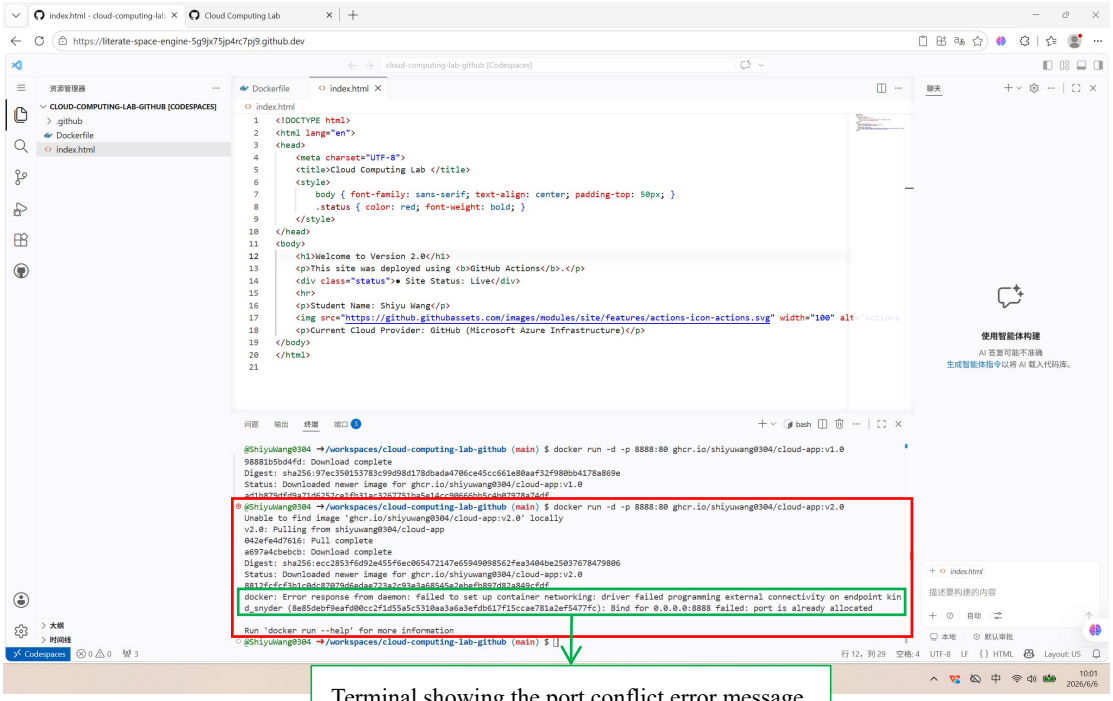
- Port 8888 was automatically forwarded
- Clicked the link to open in browser

The v1.0 application loaded successfully, displaying the original content.



5. Deploy v2.0 and Resolve Port Conflict

Attempting to run v2.0.



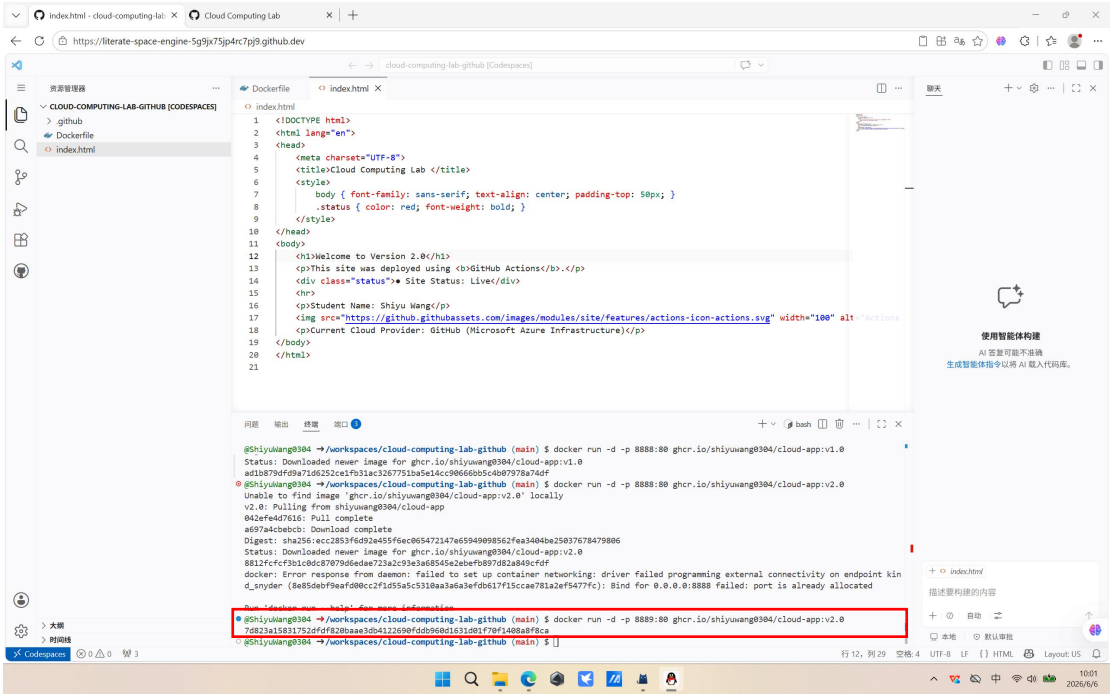
Root Cause Analysis:

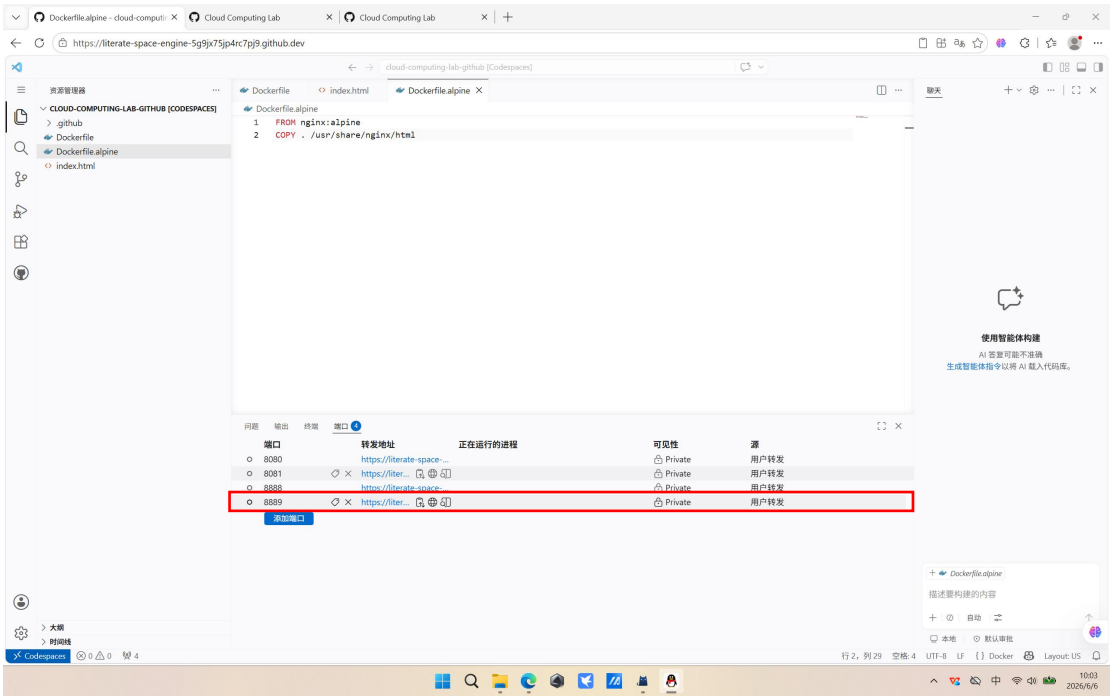
- Port 8888 on the host is already bound to the v1.0 container
- Docker cannot bind the same host port to multiple containers simultaneously
- Each container can only be accessed through unique host ports

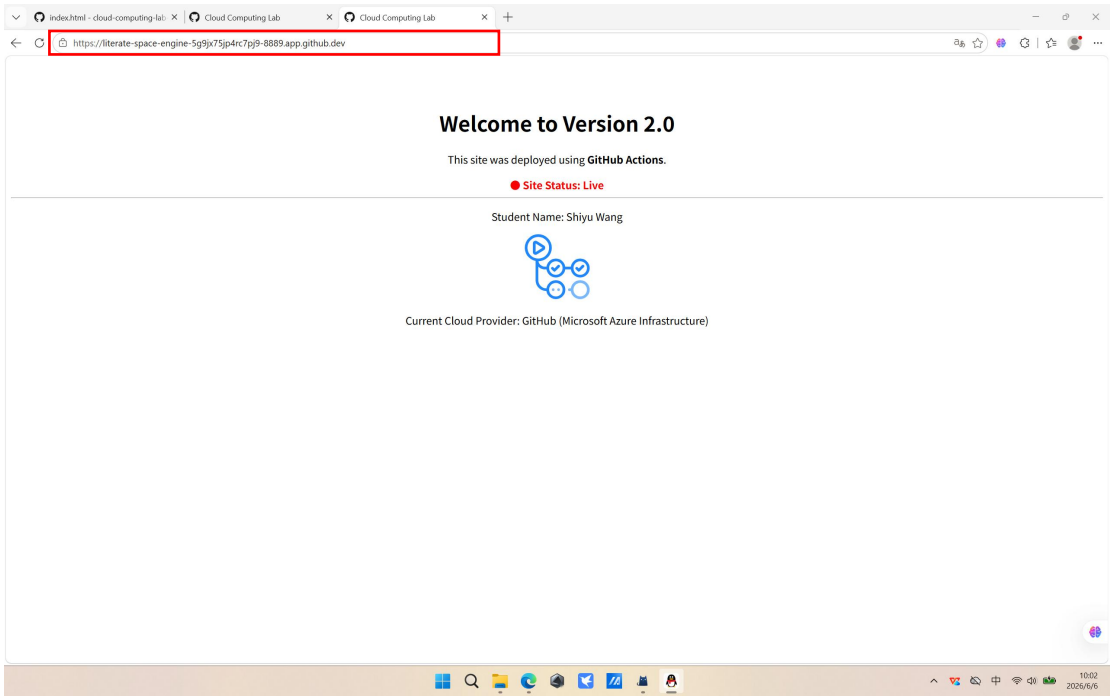
Solution Implemented:

Method 1: **Stop and Remove Existing Container**	Step 1: List running containers *docker ps* Step 2: Stop the container *docker stop container_id* Step 3: Remove the container *docker rm container_id* Step 4: Run v2.0 *docker run -d -p 8888:80 ghcr.io/shiyuwang0304/cloud-app:v2.0*
Method 2: **Use Different Port**	Run v2.0 on port 8889 while v1.0 remains on port 8888 *docker run -d -p 8889:80 ghcr.io/shiyuwang0304/cloud-app:v2.0*
Method 3: **Stop All Containers**	Stop all running containers at once *docker stop $(docker ps -q)* Run v2.0 on port 8888 *docker run -d -p 8888:80 ghcr.io/shiyuwang0304/cloud-app:v2.0*

I chose Method 2 to demonstrate proper container lifecycle management.


index.html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Cloud Computing Lab </title>
<style>
body { font-family: sans-serif; text-align: center; padding-top: 50px; }
.status { color: red; font-weight: bold; }
</style>
</head>
<body>
<h1>Welcome to Version 2.0</h1>
This site was deployed using GitHub Actions.
● Site Status: Live
<hr>
Student Name: Shiyu Wang
Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)
</body>
</html>
@ShiyuWang0304 ➜ /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/shiyuwang0304/cloud-app:v1.0
Status: Downloaded newer image for ghcr.io/shiyuwang0304/cloud-app:v1.0
@ShiyuWang0304 ➜ /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/shiyuwang0304/cloud-app:v2.0
Unable to find image 'ghcr.io/shiyuwang0304/cloud-app:v2.0' locally
v2.0: Pulling from shiyuwang0304/cloud-app
Digest: sha256:ecc2853f6d92e455f6ec065472147e6594098562fea3404be25037678479806
Status: Downloaded newer image for ghcr.io/shiyuwang0304/cloud-app:v2.0
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint kin
d_snyder (8e85debf9eafd00cc2f1d55a5c5310aa3a6a3efdb617f15ccae781a2ef5477fc): Bind for 0.0.0.0:8888 failed: port is already allocated
@ShiyuWang0304 ➜ /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8889:80 ghcr.io/shiyuwang0304/cloud-app:v2.0
7d823a15831752dfdf820baae3db4122690fddb960d1631d01f70f1408a8f8ca
@ShiyuWang0304 ➜ /workspaces/cloud-computing-lab-github (main) $


Dockerfile.alpine
FROM nginx:alpine
COPY . /usr/share/nginx/html
端口 转发地址 正在运行的进程 可见性 源
8080 https://literate-space-... Private 用户转发
8081 https://liter... Private 用户转发
8888 https://literate-space-... Private 用户转发
8889 https://liter... Private 用户转发
添加端口



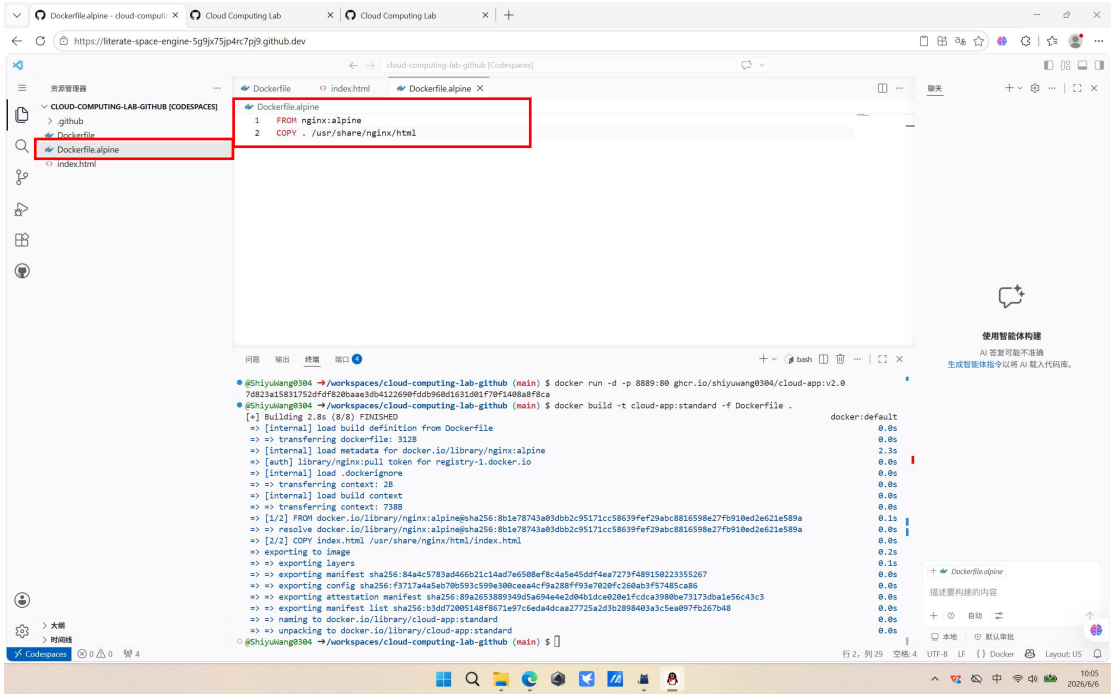
Step 6: Image Optimization - "Standard vs. Alpine"

In cloud computing, image size directly impacts:

- **Storage costs:** Cloud registries charge per GB stored
- **Deployment speed:** Larger images take longer to pull
- **Cold start time:** Serverless platforms load images on-demand
- **Network bandwidth:** Affects CI/CD pipeline speed

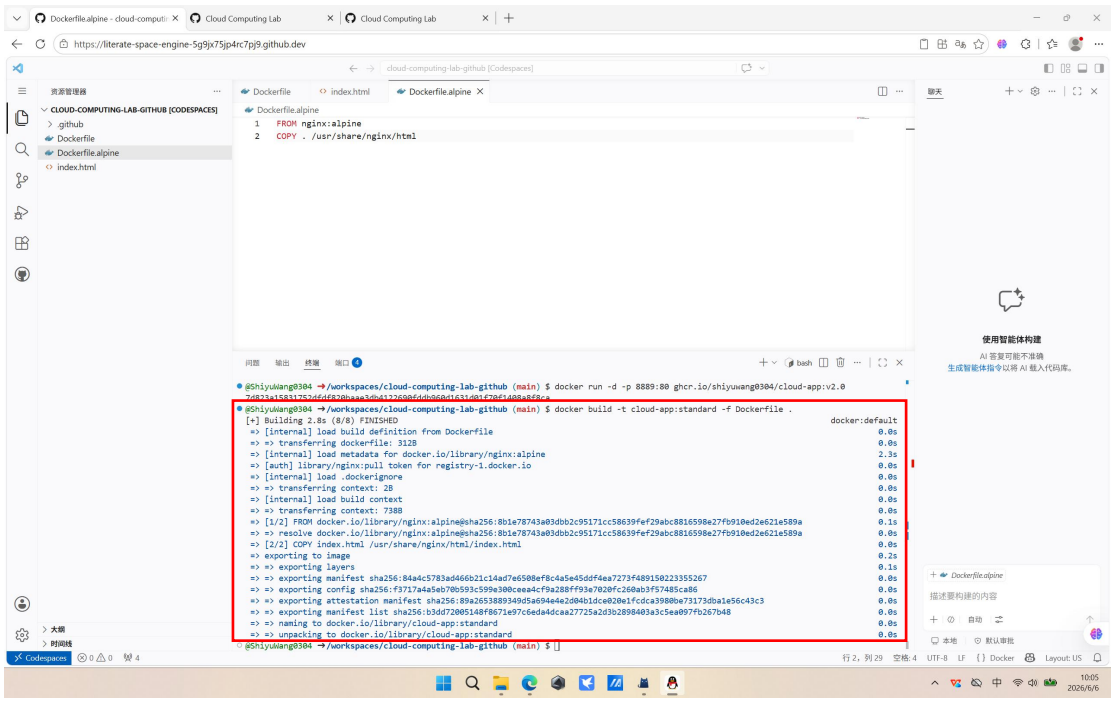
1. Create Alpine Dockerfile

I created Dockerfile.alpine with a lightweight base image.

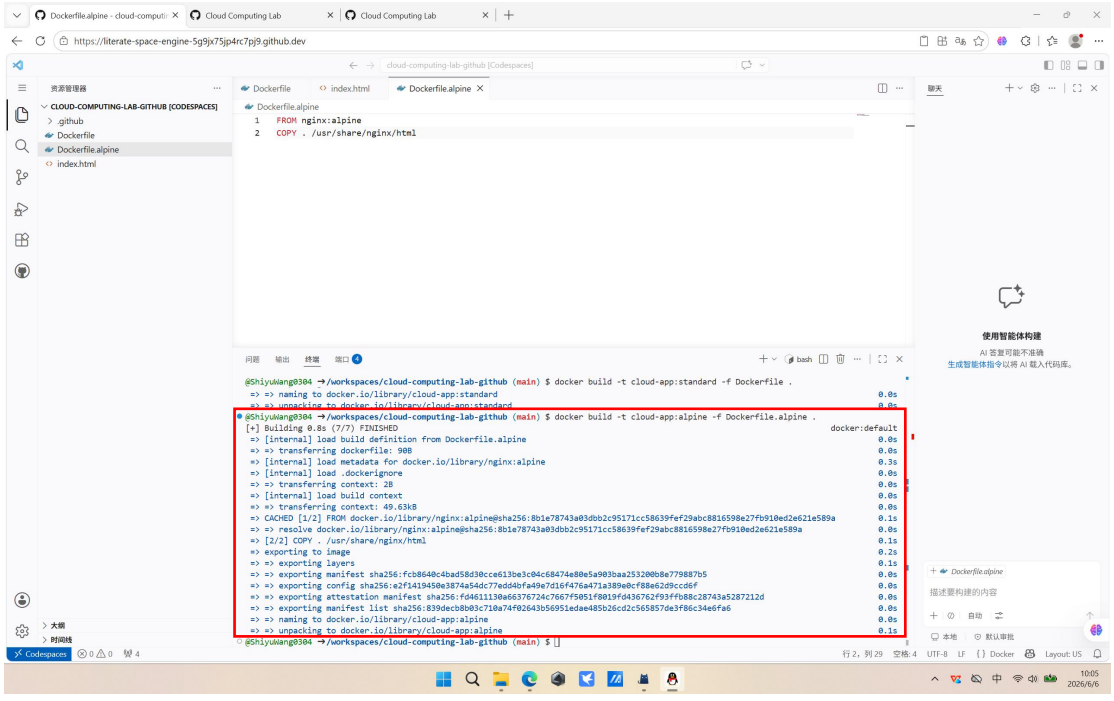


2. Build Both Versions

Build standard version (based on nginx:latest).

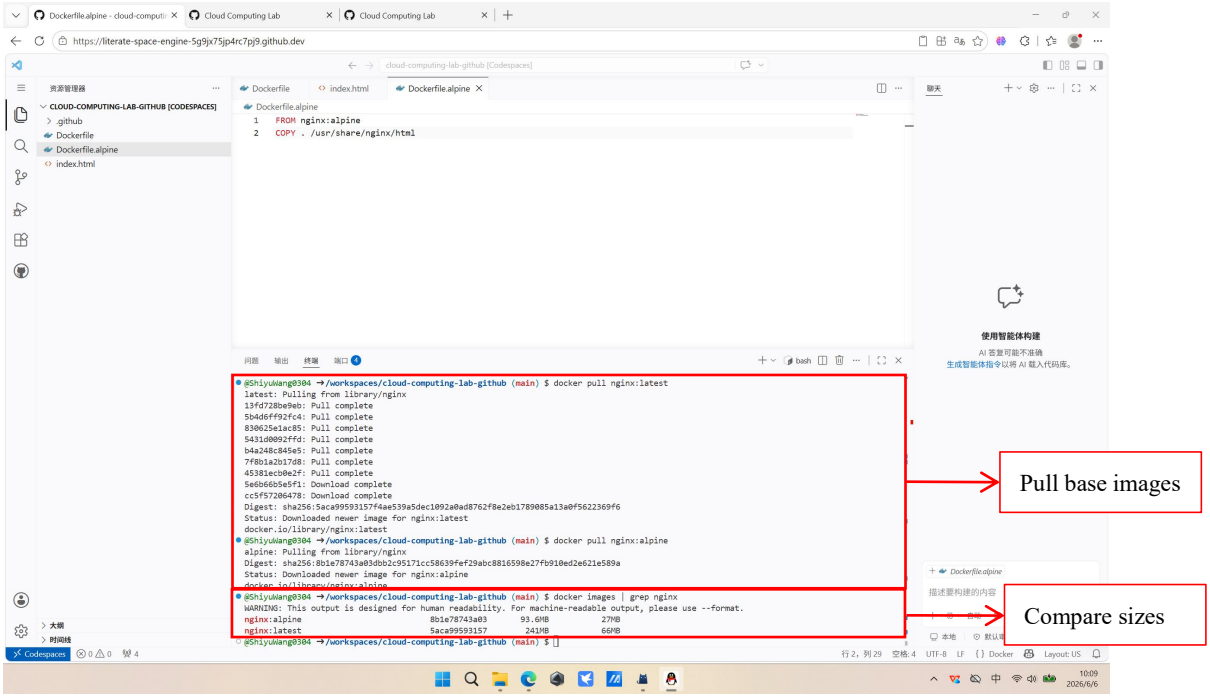


Build Alpine version (based on nginx:alpine).



3. Size Comparison

I pulled both base images and compared sizes.



Why Alpine is the Industry Standard for Production Microservices?

Reduced Attack Surface	Alpine Linux minimal installation includes only essential packages
	Fewer components = fewer potential vulnerabilities
	Smaller codebase is easier to audit and maintain
	Security patches apply faster due to smaller scope
Faster Deployment and Scaling	Image pull time: 26MB vs 145MB = ~5.5x faster download
	Cold start latency: Critical for serverless/autoscaling scenarios
	CI/CD efficiency: Faster builds and deployments
	Network bandwidth: Reduces costs in high-frequency deployments
Lower Storage Costs	Container registries charge per GB/month
	100 microservices × 118.8 MB savings = ~11.6 GB saved
	At 0.10/GB/month = 1.16/month savings (scales linearly)
	Multiplied across development, staging, and production environments
Resource Efficiency	Memory footprint: Alpine uses ~5MB RAM vs ~50MB for Debian-based
	Disk I/O: Less read/write operations during container startup
	Container density: More containers can run on the same host
Container-Native Design	Built specifically for containerized environments
	No unnecessary init systems or background services
	Optimized for single-process applications (microservices pattern)

五、Reflection

1. Problems Encountered and Proposed Solutions

During the experiment, I encountered the following challenges and successfully resolved them.

Problem	Root Cause	Solution Implemented
Port Conflict (Step 5.5)	Port 8888 was already bound to the v1.0 container. Docker cannot assign the same host port to multiple containers simultaneously.	Used docker stop and docker rm to remove the existing container before running v2.0. Alternative solutions include using a different port (-p 8889:80) or stopping all containers with docker stop $(docker ps -q).
PAT Permission Denied	Initial attempt to push images failed due to insufficient token permissions.	Regenerated the PAT with the write:packages scope selected, which includes repo and read:packages permissions.
Username Case Sensitivity	GitHub registry requires lowercase usernames in image tags. Using uppercase letters would cause push failures.	Ensured all image tags used lowercase Github username: ghcr.io/shiyuwang0304/cloud-app instead of ShiyuWang0304.
Understanding Layer Reuse	Initially unclear why some layers showed "Layer already exists" while others were uploaded.	Analyzed the push output and understood that Docker uses content-addressable storage—layers are identified by SHA256 hash of their content. Only changed layers are uploaded.

2. Efficiency: Why did Step 4.3 (Pushing v2.0) upload much faster than Step 3.1? Relate this to the concept of Layer Caching.

1) Observation

Step 3.1 (Pushing v1.0): All layers were uploaded to GHCR

Step 4.3 (Pushing v2.0): Most layers displayed "Layer already exists" and were skipped

2) Technical Explanation

Docker images are built using a layered architecture based on a union filesystem. Each instruction in a Dockerfile creates a separate, read-only layer. This design enables Layer Caching.

(1) Content-Addressable Storage

Each layer is identified by a unique SHA256 hash derived from its content.

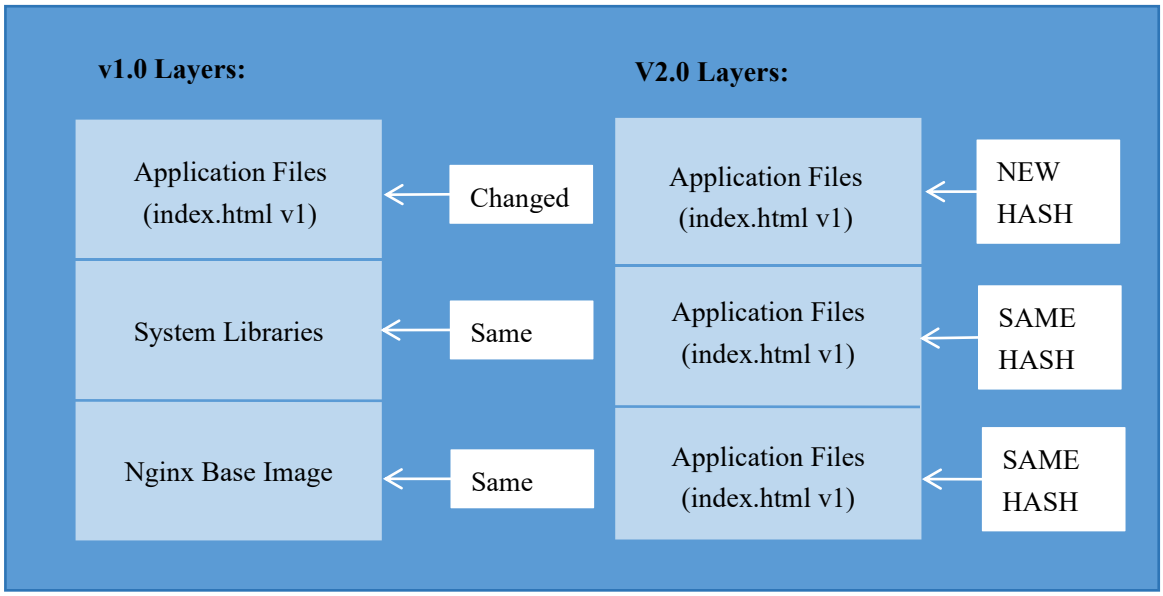
When pushing v2.0, Docker performs the following comparison:

Layer	Content	Hash	v1.0 Push	v2.0 Push
Base Image	Nginx OS files	Unique_id	Uploaded	**Skipped** (hash matches)
System Libraries	Dependencies	Unique_id	Uploaded	**Skipped** (hash matches)

Application Files	index.html	Changed	Uploaded	**Uploaded** (new hash)

(2) Incremental Update Mechanism

Between v1.0 and v2.0, only index.html was modified.



Result:

- v1.0 Push: 3 layers uploaded
- v2.0 Push: 1 layer uploaded
- Time saved: ~95% of upload time

(3) Benefits of Layer Caching

Benefit	Description
Bandwidth Efficiency	Only changed layers are transferred over the network
Storage Optimization	Layers are shared across multiple image versions, reducing registry storage costs
Faster Deployments	CI/CD pipelines can reuse cached layers, accelerating build and deployment times
Version Control	Enables efficient management of multiple application versions

The dramatic speed improvement in Step 4.3 demonstrates Docker's intelligent layer management system. By reusing unchanged layers, Docker minimizes network transfer and storage requirements, making containerized applications highly efficient for iterative development and deployment.

3. Security: Why is using a Personal Access Token (PAT) more secure than using your actual GitHub password in a terminal?

Using a Personal Access Token (PAT) instead of a plaintext password provides multiple layers of security protection.

1) Principle of Least Privilege

- **GitHub Password:** Full access to entire account (all repositories, settings, billing).
- **PAT with write:packages:** Limited to specific permissions (only push to container registry).

If a PAT is compromised, the attacker's access is restricted to only the granted scopes, whereas a compromised password provides full account access.

2) Token Revocability Without Password Change

Scenario	Using Password	Using PAT
Token compromised	Must change password everywhere	Simply revoke the specific PAT
Need to revoke access	Disrupts all services	Generate new PAT, revoke old one
Audit access logs	Cannot distinguish sources	Each PAT has unique identifier

3) Prevention of Credential Exposure in Command History

(1) Insecure Method (Plaintext Password):

docker login ghcr.io -u shiyuwang0304 -p MyActualPassword123!

(2) Secure Method (PAT via Environment Variable):

The --password-stdin flag ensures the credential is passed via standard input rather than command-line arguments, preventing it from appearing in process listings (ps aux) or shell history.

4) Audit Trail and Accountability

GitHub provides detailed logs for each PAT:

- **Creation timestamp:** When the token was generated
- **Last used timestamp:** Most recent API call
- **Expiration date:** Automatic invalidation after specified period
- **Permissions granted:** Clear record of access scope

This enables security teams to track credential usage and identify potential breaches.